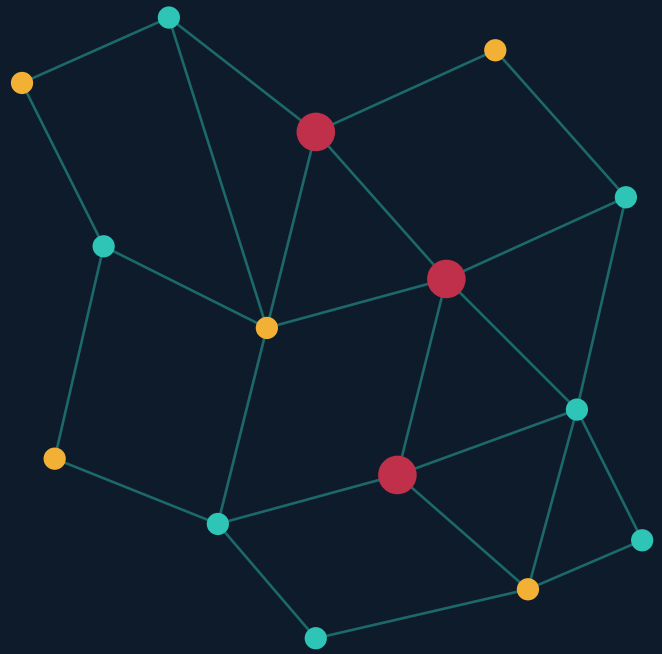
ΕΒΔΟΜΑΔΑ 5

Ερωτήματα μέσα σε ερωτήματα

Υποερωτήματα · IN · EXISTS · ANY · ALL · Views · Star & snowflake schema



Δρ. Αθανάσιος Κρανιάς
Βάσεις Δεδομένων & Επιχειρηματική Αναλυτική · Τμήμα Χρηματοοικονομικής και Τραπεζικής Διοικητικής · Πανεπιστήμιο Πειραιώς

Πού είμαστε

Η τελευταία εβδομάδα της ενότητας SQL — από την επόμενη, οικονομετρία

Εβδομάδα 4 σε 30″

- INNER: μόνο ζευγάρια · LEFT: όλα από αριστερά
- LEFT JOIN + IS NULL = «τι λείπει»
- Πολλαπλά JOINs: Financials → Companies → Sectors
- COUNT πριν και μετά το JOIN

Σήμερα θα μάθεις να:

- χρησιμοποιείς το αποτέλεσμα ενός ερωτήματος μέσα σε άλλο
- απαντάς «υπάρχει;» με EXISTS
- συγκρίνεις με ομάδες τιμών (ANY, ALL)
- αποθηκεύεις ερωτήματα ως Views για αναφορές

Μια ερώτηση, δύο βήματα

«Ποιες εταιρείες έχουν περισσότερους υπαλλήλους από τον μέσο όρο της αγοράς;»

```sql
-- Βήμα 1: ποιος είναι ο μέσος όρος;
SELECT AVG(Employees * 1.0)
FROM Companies;
-- 2969.565217
```

```sql
-- Βήμα 2: αντιγράφω τον αριθμό
SELECT Ticker, Employees
FROM Companies
WHERE Employees > 2969.57;
```



Το πρόβλημα: ο αριθμός είναι «καρφωμένος». Αν προστεθεί μια εταιρεία, ο μέσος όρος αλλάζει — και το ερώτημα δίνει λάθος αποτέλεσμα χωρίς να το καταλάβει κανείς.

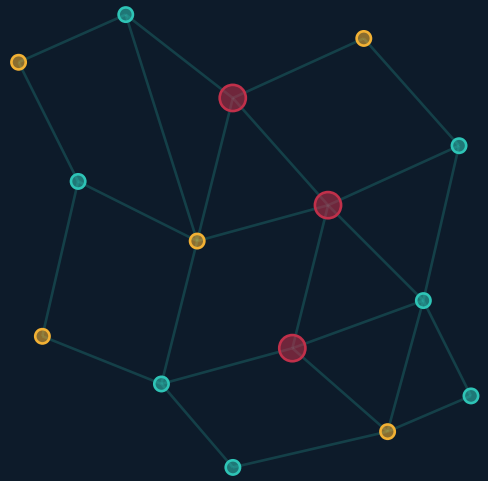
✓ ΜΕ ΥΠΟΕΡΩΤΗΜΑ: ΕΝΑ ΒΗΜΑ, ΠΑΝΤΑ ΣΩΣΤΟ

```sql
SELECT Ticker, Employees
FROM Companies
WHERE Employees > (SELECT AVG(Employees * 1.0) FROM Companies);
```

01

Υποερωτήματα

Subqueries: ένα SELECT μέσα σε παρενθέσεις



Υποερώτημα που επιστρέφει μία τιμή

Scalar subquery: εκτελείται πρώτο, και το αποτέλεσμά του μπαίνει στη θέση του

```sql
SELECT Ticker, Employees
FROM Companies
WHERE Employees >
      (SELECT AVG(Employees * 1.0)
       FROM Companies)
ORDER BY Employees DESC;
```

Πρώτα τρέχει το εσωτερικό ερώτημα → 2969,57

Μετά το εξωτερικό: WHERE Employees > 2969,57

Πρέπει να επιστρέφει ακριβώς μία γραμμή, μία στήλη

Ticker	Employees
AGMR	11200
AELT	9800
ATLB	8200
ESPR	5600
OLYE	5400
TTNC	4300
KRNB	3100

7 από τις 23 εταιρείες με διαθέσιμο αριθμό υπαλλήλων

Πού μπαίνει ένα υποερώτημα;

Σε τρία σημεία — με διαφορετικό ρόλο το καθένα

WHERE — Ως φίλτρο

```
WHERE Employees >
  (SELECT AVG(…) …)
```

Σύγκριση με τιμή ή λίστα τιμών

SELECT — Ως νέα στήλη

```
SELECT Ticker, Employees,
  (SELECT AVG(…) …)
  AS MarketAvg
```

Ίδια τιμή σε κάθε γραμμή

FROM — Ως «προσωρινός» πίνακας

```
SELECT …
FROM (SELECT …) AS t
```

Derived table — πρέπει να έχει ψευδώνυμο

Υποερώτημα στο FROM (derived table)

«Πρώτα υπολόγισε, μετά συνόψισε»: μέσο ROE ανά κλάδο, 2025

```
SELECT s.SectorName,
       AVG(t.ROE) * 100 AS AvgROEPct
FROM (SELECT CompanyID,
             NetIncome / Equity AS ROE
      FROM Financials
      WHERE FiscalYear = 2025) AS t
JOIN Companies c ON c.CompanyID = t.CompanyID
JOIN Sectors   s ON s.SectorID  = c.SectorID
GROUP BY s.SectorName
ORDER BY AvgROEPct DESC;
```

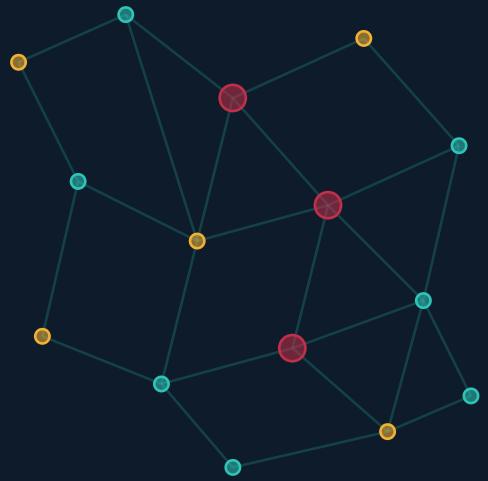
SectorName	ROE %
Τράπεζες	24.0
Τεχνολογία	22.6
Ενέργεια	22.5
Ναυτιλία	19.1
Τουρισμός	16.6
Λιανεμπόριο	16.3
Τηλεπικοινωνίες	13.3
Κατασκευές	8.5

Μέσος όρος των ROE των εταιρειών (στρογγ.) · η Ikaros Cloud (NULL) εξαιρείται αυτόματα

02

IN, EXISTS, ANY, ALL

Υποερωτήματα που επιστρέφουν πολλές τιμές



IN με υποερώτημα

Το υποερώτημα επιστρέφει μια λίστα — το IN ελέγχει αν η τιμή είναι μέσα

```sql
SELECT Ticker, CompanyName
FROM Companies
WHERE SectorID IN (
    SELECT SectorID
    FROM Sectors
    WHERE SectorName IN (N'Τεχνολογία', N'Ναυτιλία')
);
```

Ticker	CompanyName
ATHS	Athena Software
DLPA	Delphi Analytics
HRMS	Hermes Systems
IKRC	Ikaros Cloud
PSDS	Poseidon Shipping
NRSM	Nereus Maritime
ARGL	Argo Lines

Υποερώτημα → (3, 4) · εξωτερικό → WHERE SectorID IN (3, 4)

Ισοδύναμο με JOIN Companies–Sectors · συχνά πιο ευανάγνωστο όταν θέλουμε μόνο στήλες του ενός πίνακα

NOT IN — προσοχή αν η λίστα περιέχει NULL: δεν επιστρέφει τίποτα!

EXISTS: «υπάρχει τουλάχιστον μία γραμμή;»

Συσχετισμένο υποερώτημα (correlated): εκτελείται για κάθε γραμμή του εξωτερικού

```sql
-- Εταιρείες που έχουν δώσει μέρισμα
SELECT c.Ticker, c.CompanyName
FROM Companies AS c
WHERE EXISTS (
    SELECT 1
    FROM Dividends AS d
    WHERE d.CompanyID = c.CompanyID
);
-- 16 εταιρείες
```

```sql
-- Εταιρείες που ποτέ δεν είχαν ζημιές
SELECT c.Ticker
FROM Companies AS c
WHERE NOT EXISTS (
    SELECT 1 FROM Financials AS f
    WHERE f.CompanyID = c.CompanyID
      AND f.NetIncome < 0
);
-- 20 εταιρείες
```

`c.CompanyID` μέσα στο υποερώτημα = η σύνδεση με την τρέχουσα γραμμή. **`SELECT 1:`** δεν μας νοιάζει τι επιστρέφει — μόνο αν επιστρέφει κάτι.

ANY & ALL: σύγκριση με μια ομάδα τιμών

Έσοδα 2025: πώς συγκρίνονται οι εταιρείες με τις τράπεζες και την τεχνολογία;

> ANY: ΜΕΓΑΛΥΤΕΡΑ ΑΠΟ ΤΟΥΛΑΧΙΣΤΟΝ ΜΙΑ ΤΡΑΠΕΖΑ

```
SELECT c.Ticker, f.Revenue
FROM Financials f
JOIN Companies c ON c.CompanyID = f.CompanyID
WHERE f.FiscalYear = 2025
  AND c.SectorID <> 1
  AND f.Revenue > ANY (
      SELECT f2.Revenue FROM Financials f2
      JOIN Companies c2
        ON c2.CompanyID = f2.CompanyID
      WHERE c2.SectorID = 1
        AND f2.FiscalYear = 2025);
```

Τράπεζα	Έσοδα 2025
PGSF	1874.28
ATLB	1047.57
KRNB	825.94

Τελεστής	Ισοδύναμο	Γραμμές
> ANY (τράπεζες)	> MIN = 825.94	8
> ALL (τράπεζες)	> MAX = 1874.28	4
> ALL (τεχνολογία)	> MAX = 331.35	15

Ποσά σε εκατ. € · ANY = SOME

Συσχετισμένο υποερώτημα

Σύγκριση με τον «δικό σου» μέσο όρο: εταιρείες μεγαλύτερες από τον μέσο όρο του κλάδου τους

```sql
SELECT c.Ticker, c.SectorID, c.Employees
FROM Companies AS c
WHERE c.Employees > (
    SELECT AVG(c2.Employees * 1.0)
    FROM Companies AS c2
    WHERE c2.SectorID = c.SectorID   -- ο κλάδος της c
)
ORDER BY c.SectorID;
```

Ticker	Sector	Employees
ATLB	1	8200
OLYE	2	5400
HRMS	3	1900
PSDS	4	1200
CYCH	5	2600
MNRS	5	1850
AELT	6	9800
TTNC	7	4300
AGMR	8	11200



Η PSDS (1.200) δεν είναι «μεγάλη» στην αγορά — αλλά είναι η μεγαλύτερη της ναυτιλίας.

JOIN ή υποερώτημα;

Συχνά λύνουν το ίδιο πρόβλημα — διαλέγουμε ό,τι διαβάζεται πιο εύκολα

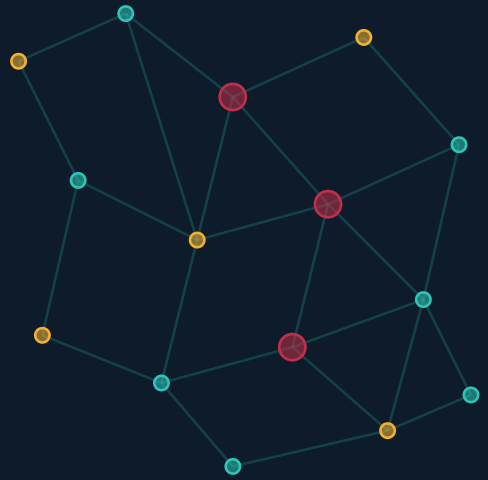
Θέλω…	Προτιμώ
Στήλες από δύο ή περισσότερους πίνακες στο αποτέλεσμα	JOIN
Να φιλτράρω με βάση μια συνολική τιμή (μέσος όρος, max)	Υποερώτημα στο WHERE
Να ελέγξω «υπάρχει / δεν υπάρχει» σχετική εγγραφή	EXISTS / NOT EXISTS
Να συνοψίσω δεδομένα που πρώτα πρέπει να μετασχηματίσω	Υποερώτημα στο FROM
Να ξαναχρησιμοποιώ το ίδιο ερώτημα σε πολλές αναφορές	VIEW (επόμενη ενότητα)

Ο SQL Server συχνά μετατρέπει εσωτερικά το ένα στο άλλο — η απόδοση σπάνια είναι το κριτήριο σε αυτό το επίπεδο.

03

Views: αποθηκευμένα ερωτήματα

Εικονικοί πίνακες για reporting — η γέφυρα προς την R και το Power BI



Τι είναι ένα View;

Ένα αποθηκευμένο SELECT με όνομα — συμπεριφέρεται σαν πίνακας, αλλά δεν αποθηκεύει δεδομένα

```
CREATE VIEW vw_CompanyProfile AS
SELECT c.CompanyID, c.Ticker, c.CompanyName,
       s.SectorName, c.Country, c.Employees
FROM Companies AS c
JOIN Sectors   AS s ON s.SectorID = c.SectorID;
GO

-- Από εδώ και πέρα: σαν να ήταν πίνακας
SELECT * FROM vw_CompanyProfile
WHERE SectorName = N'Ναυτιλία';
```



Απλότητα
Το JOIN γράφεται μία φορά — οι χρήστες βλέπουν έναν «καθαρό» πίνακα



Συνέπεια
Όλες οι αναφορές χρησιμοποιούν τον ίδιο ορισμό (π.χ. του ROE)



Ασφάλεια
Δίνουμε πρόσβαση στο view, όχι στους πίνακες με όλες τις στήλες

Κανόνες για Views στον SQL Server

Τρεις λεπτομέρειες που θα συναντήσετε στο SSMS

GO πριν και μετά

Το CREATE VIEW πρέπει να είναι μόνο του στο «batch». Χωρίς GO → σφάλμα «CREATE VIEW must be the only statement in the batch».

```
GO
CREATE VIEW …
GO
```

Χωρίς ORDER BY

Ένα view είναι «πίνακας» — οι πίνακες δεν έχουν σειρά. Η ταξινόμηση μπαίνει στο SELECT που διαβάζει το view.

```
SELECT * FROM vw_…
ORDER BY …;
```

Αλλαγή & διαγραφή

CREATE OR ALTER ενημερώνει τον ορισμό χωρίς να χρειαστεί DROP. DROP VIEW το σβήνει — τα δεδομένα των πινάκων μένουν ανέγγιχτα.

```
CREATE OR ALTER VIEW …
DROP VIEW vw_…;
```

Απλά ή «υλοποιημένα» (materialized) views;

Αποθηκεύουμε το ερώτημα — ή και το αποτέλεσμά του;

	Απλό view (αυτό που φτιάχνουμε)	Materialized view
Τι αποθηκεύεται	Μόνο το SELECT	Το SELECT και οι γραμμές του αποτελέσματος
Πότε τρέχει το ερώτημα	Κάθε φορά που διαβάζουμε το view	Μία φορά — μετά ξαναδιαβάζεται έτοιμο
Ταχύτητα	Όσο το ερώτημα	Πολύ γρήγορο σε βαριά ερωτήματα
Ενημέρωση	Πάντα τα τρέχοντα δεδομένα	Χρειάζεται ανανέωση (refresh)
Χώρος στον δίσκο	Σχεδόν μηδέν	Όσο και το αποτέλεσμα



Στον SQL Server το αντίστοιχο λέγεται indexed view (WITH SCHEMABINDING + UNIQUE CLUSTERED INDEX) και ενημερώνεται αυτόματα. Στο DataCamp (PostgreSQL): CREATE MATERIALIZED VIEW … και REFRESH MATERIALIZED VIEW. Για τα δεδομένα μας, τα απλά views αρκούν.

Ένα view για reporting: η απόδοση του 2025

Συνδυάζει όλα όσα μάθαμε: JOINs, υποερωτήματα, υπολογιζόμενες στήλες

```sql
CREATE OR ALTER VIEW vw_Performance2025 AS
SELECT c.Ticker, s.SectorName,
       b.ClosePrice AS PriceStart,
       e.ClosePrice AS PriceEnd,
       100.0 * (e.ClosePrice / b.ClosePrice - 1)
             AS ReturnPct
FROM Companies c
JOIN Sectors s ON s.SectorID = c.SectorID
JOIN StockPrices b ON b.CompanyID = c.CompanyID
  AND b.TradeDate = (SELECT MIN(TradeDate) FROM StockPrices
                     WHERE TradeDate >= '2025-01-01')
JOIN StockPrices e ON e.CompanyID = c.CompanyID
  AND e.TradeDate = (SELECT MAX(TradeDate) FROM StockPrices);
```

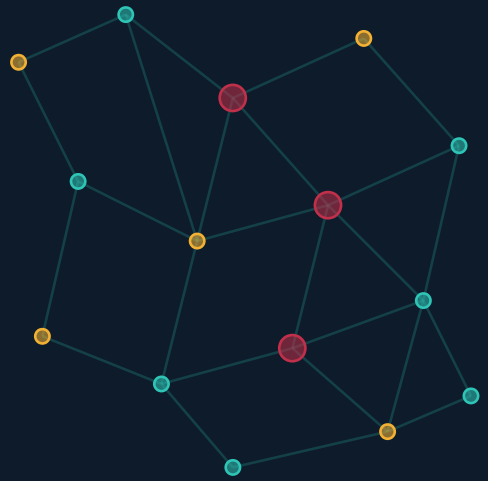
Ticker	Return %
IKRC	+123.0
KRNB	+106.8
AGMR	+63.9
…	…
OLYE	−20.8
ATLB	−25.0

Δείκτης αγοράς 2025: +9,2%. 16 από 24 μετοχές έκλεισαν θετικά.

04

Από τις συναλλαγές στην ανάλυση

OLTP & OLAP · fact & dimension tables · star & snowflake



OLTP ή OLAP;

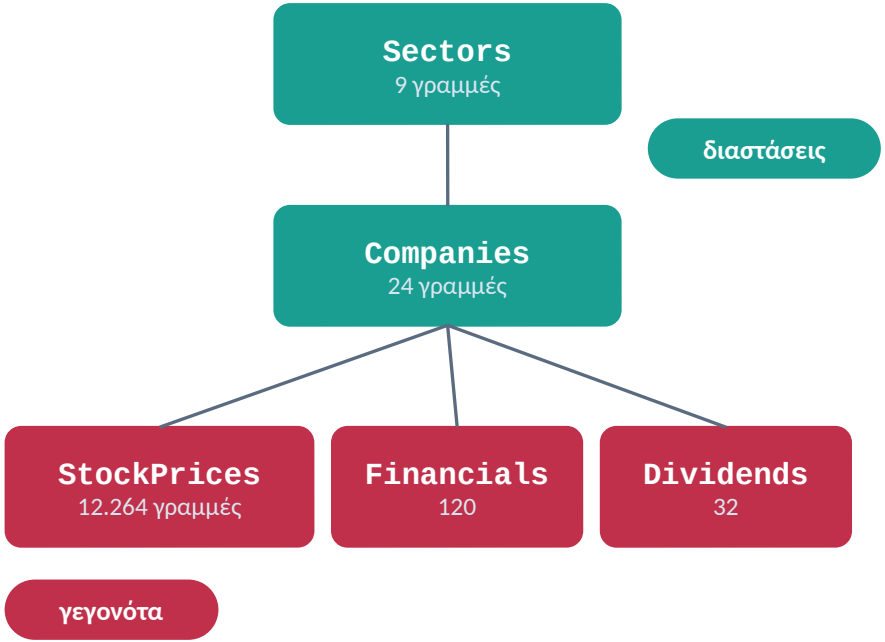
Δύο τρόποι να οργανώσεις μια βάση — ανάλογα με το ποιος τη χρησιμοποιεί και για ποιο λόγο

	OLTP · βάση συναλλαγών	OLAP · βάση ανάλυσης
Τα αρχικά	Online Transaction Processing = επεξεργασία συναλλαγών σε πραγματικό χρόνο	Online Analytical Processing = αναλυτική επεξεργασία δεδομένων
Σκοπός	Καταχώρηση των καθημερινών συναλλαγών	Ανάλυση και λήψη αποφάσεων
Παράδειγμα	Core banking, σύστημα εντολών χρηματιστηρίου	Αποθήκη δεδομένων (data warehouse), BI
Τυπικές εντολές	Πολλά μικρά INSERT / UPDATE	Λίγα μεγάλα SELECT με JOIN & GROUP BY
Δεδομένα	Η τρέχουσα κατάσταση	Ιστορικό ετών
Σχεδιασμός	Κανονικοποιημένος (πολλοί μικροί πίνακες)	Star ή snowflake schema
Χρήστες	Εφαρμογές, ταμίες, πελάτες	Αναλυτές, διοίκηση, dashboards

Η StockMarketDB είναι μια μικρή βάση ανάλυσης: ιστορικό δύο ετών, φορτωμένο μία φορά — εμείς μόνο τη ρωτάμε με SELECT.

Πίνακες γεγονότων & πίνακες διαστάσεων

Fact tables: τι μετράμε · Dimension tables: ποιον, τι, πότε — στη δική μας βάση



Fact table

Αριθμοί που μετράμε (τιμές, όγκοι, έσοδα, μερίσματα) + ξένα κλειδιά. Πολλές γραμμές — μεγαλώνει κάθε μέρα.

Dimension table

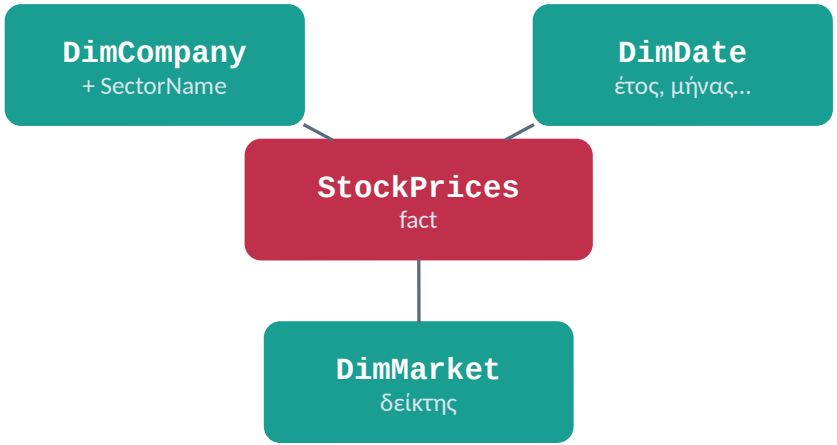
Περιγραφές: ποια εταιρεία, ποιος κλάδος, ποια χώρα. Λίγες γραμμές — αλλάζει σπάνια.

Κανόνας: τα facts τα αθροίζουμε (SUM, AVG) · με τις διαστάσεις φιλτράρουμε και ομαδοποιούμε (WHERE, GROUP BY).

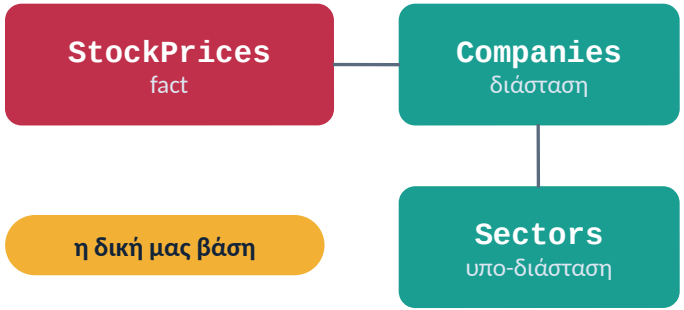
Star ή snowflake schema;

Δύο τρόποι να «δέσεις» τις διαστάσεις γύρω από τα γεγονότα

Star (αστέρι)



Snowflake (χιονονιφάδα)

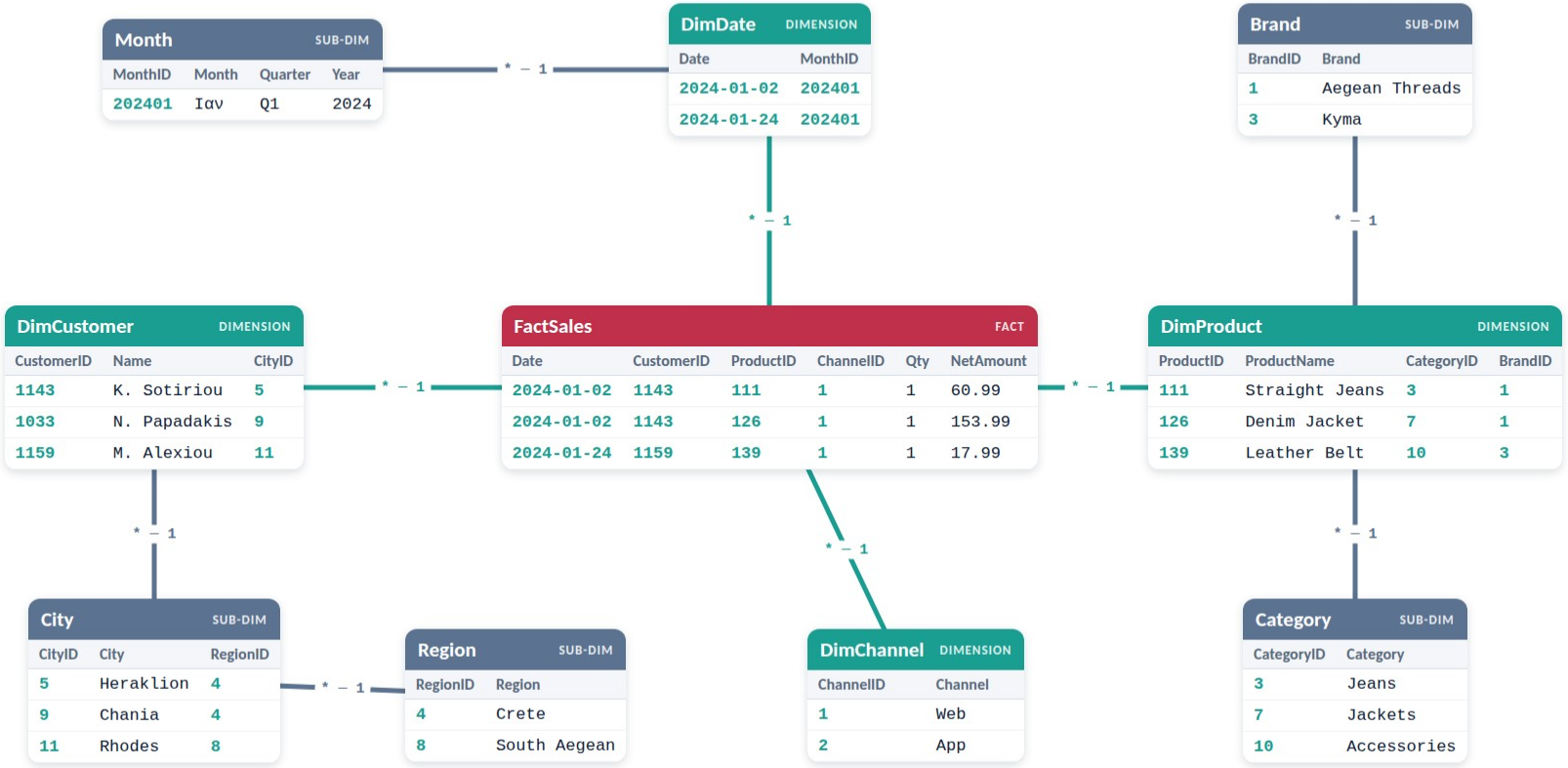


	Star	Snowflake
JOINs για ένα ερώτημα	Λίγα — ένα ανά διάσταση	Περισσότερα (αλυσίδες)
Επανάληψη κειμένου	Ναι (ο κλάδος σε κάθε εταιρεία)	Όχι — κανονικοποιημένο
Πού ταιριάζει	BI, Power BI, dashboards	Μεγάλες, «μαζεμένες» διαστάσεις

Snowflake schema · UniStyle eshop

χιονονιφάδα · κανονικοποιημένο

10 πίνακες. Οι διαστάσεις «σπάνε» σε υπο-διαστάσεις · «πωλήσεις ανά περιφέρεια» = FactSales → DimCustomer → City → Region (3 JOIN)

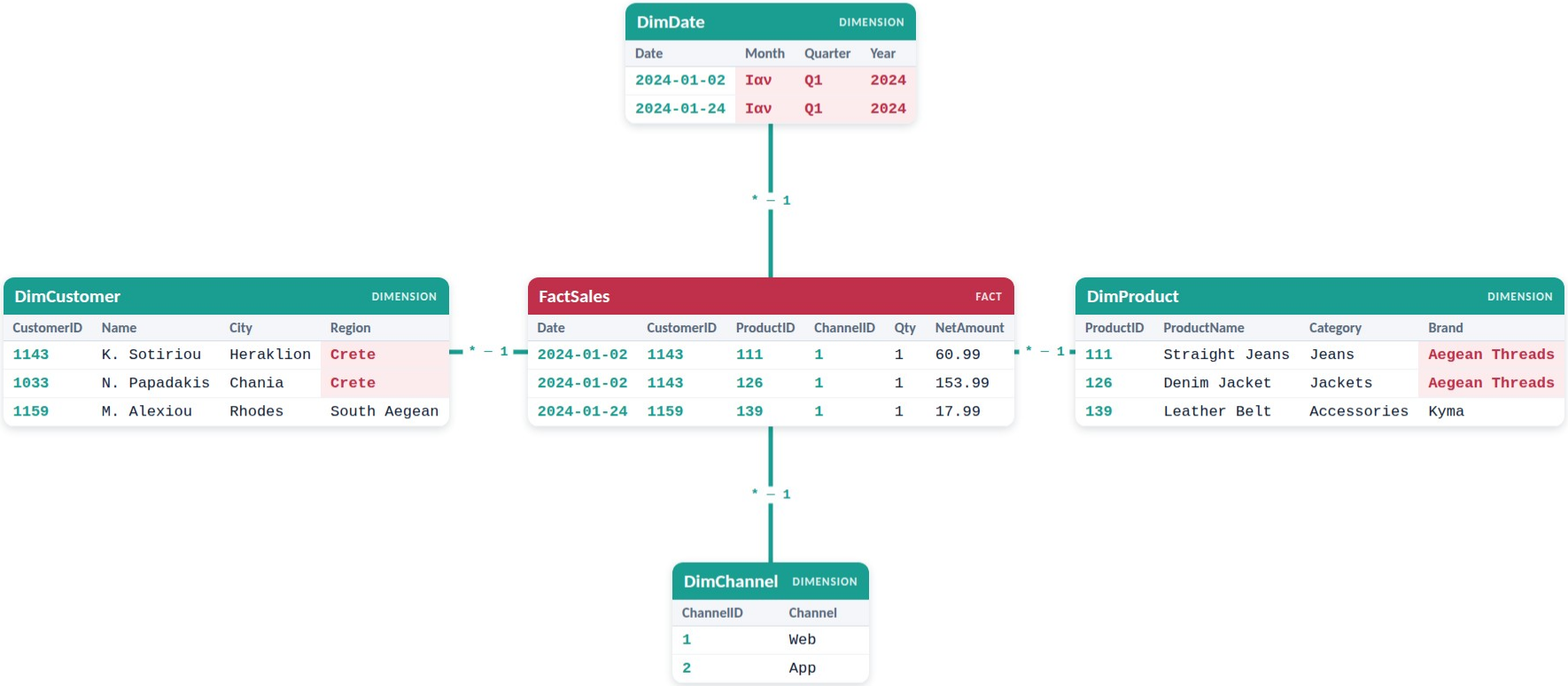


Πίνακας γεγονότων (fact): τα νούμερα · Διάσταση: ποιος · τι · πότε · Υπο-διάσταση (μόνο στο snowflake) · Τιμή που επαναλαμβάνεται

Star schema · UniStyle eshop

αστέρι · για ανάλυση

5 πίνακες. Κάθε διάσταση απέχει 1 JOIN από το κέντρο · «πωλήσεις ανά περιφέρεια» = FactSales → DimCustomer



Πίνακας γεγονότων (fact): τα νούμερα · Διάσταση: ποιος · τι · πότε · Υπο-διάσταση (μόνο στο snowflake) · Τιμή που επαναλαμβάνεται

Κανονικοποίηση σε μία διαφάνεια

Γιατί η βάση μας έχει έξι πίνακες και όχι έναν μεγάλο «Excel»

Ένας μεγάλος πίνακας (μη κανονικοποιημένος)

Ticker	CompanyName	SectorName	TradeDate	Close
ATLB	Atlas Bank	Τράπεζες	2025-12-30	1.38
ATLB	Atlas Bank	Τράπεζες	2025-12-31	1.38
KRNB	Kronos Bancorp	Τράπεζες	2025-12-31	12.24
…	…	…	…	…

Ενημέρωση: αλλάζει το όνομα ενός κλάδου → 12.264 αλλαγές

Εισαγωγή: νέος κλάδος χωρίς εταιρείες (Βιοτεχνολογία) → δεν «χωράει»

Διαγραφή: σβήνουμε μια εταιρεία → χάνεται και ο κλάδος

1NF — **Μία τιμή ανά κελί**
όχι «ATLB, KRNB» σε ένα πεδίο

2NF — **Κάθε στήλη εξαρτάται από όλο το κλειδί**
το όνομα εταιρείας δεν εξαρτάται από την ημερομηνία → Companies

3NF — **Καμία στήλη δεν εξαρτάται από άλλη μη-κλειδί**
SectorName εξαρτάται από SectorID → Sectors

Κανονικοποίηση = κάθε πληροφορία γράφεται μία φορά. Το τίμημα: περισσότερα JOINs.

Views: από snowflake σε star

Δεν αλλάζουμε τους πίνακες — φτιάχνουμε τις διαστάσεις ως views

```sql
-- Διάσταση εταιρείας: Companies + Sectors σε ένα
CREATE OR ALTER VIEW vw_DimCompany AS
SELECT c.CompanyID, c.Ticker, c.CompanyName,
       s.SectorName, c.Country
FROM   Companies c
JOIN   Sectors s ON s.SectorID = c.SectorID;
GO
```

24 γραμμές

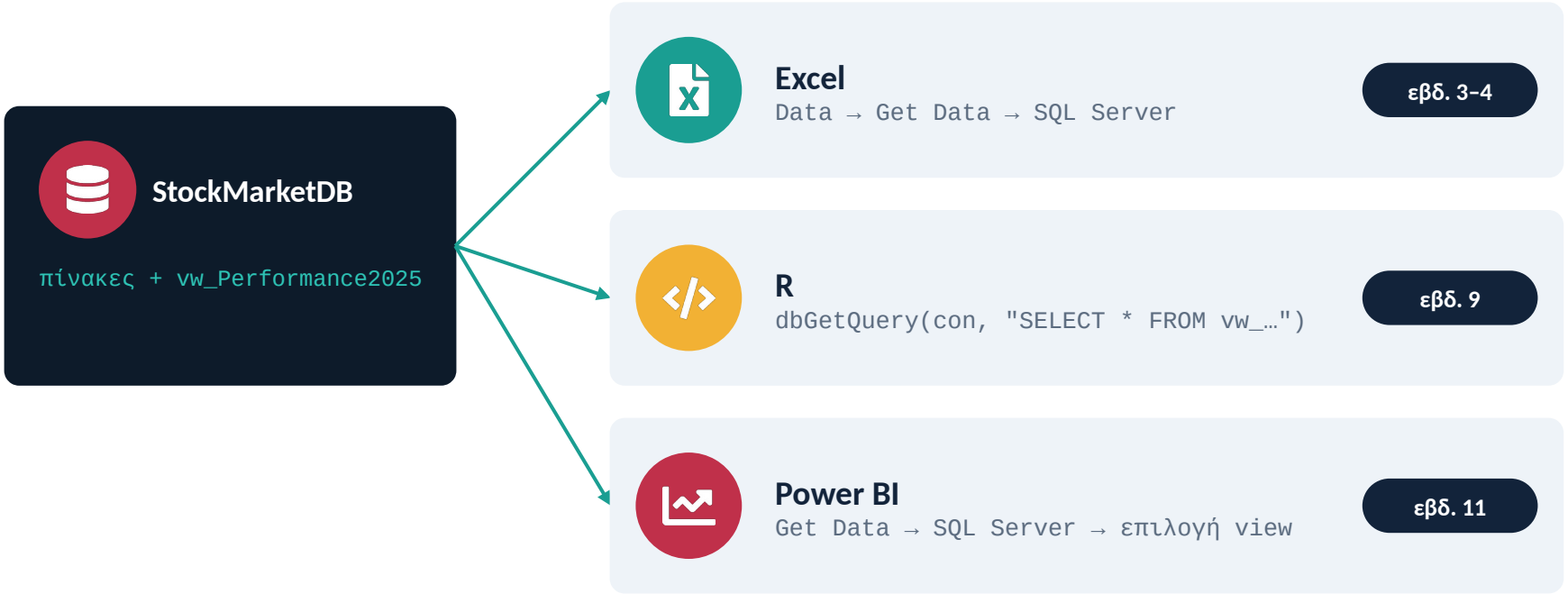
```sql
-- Διάσταση ημερομηνίας
CREATE OR ALTER VIEW vw_DimDate AS
SELECT TradeDate AS DateKey,
       YEAR(TradeDate)  AS [Year],
       DATEPART(QUARTER, TradeDate) AS [Quarter],
       MONTH(TradeDate) AS [Month],
       DATENAME(WEEKDAY, TradeDate) AS [Weekday]
FROM   MarketIndex;
GO
```

511 γραμμές — μία για κάθε ημέρα συναλλαγών

Εβδ. 11 · Power BI: φορτώνουμε StockPrices + vw_DimCompany + vw_DimDate → ένα καθαρό star schema στο Model view.

Γιατί τα Views μετράνε για το υπόλοιπο εξάμηνο

Το ίδιο view τροφοδοτεί όλα τα εργαλεία — ένας ορισμός, μία «αλήθεια»

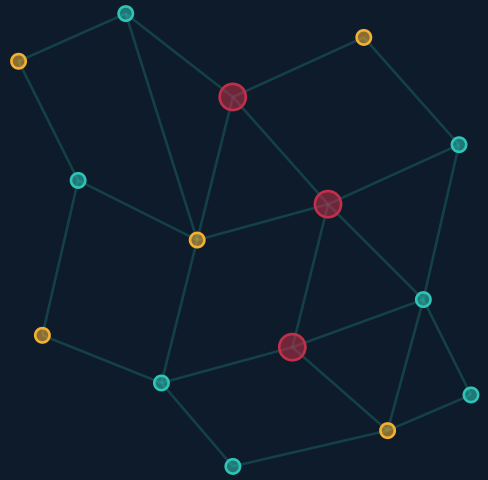


Τέλος ενότητας SQL ✓

05

Εργαστήριο

Υποερωτήματα και τα πρώτα σας views



Άσκηση 1 · Υποερωτήματα

40 λεπτά · ένα ερώτημα για κάθε ερώτηση

Q1 Μετοχές με τιμή κλεισίματος 31/12/2025 πάνω από τον μέσο όρο όλων των μετοχών εκείνη την ημέρα

Q2 Εταιρείες του κλάδου «Τουρισμός» — χωρίς να γράψετε το SectorID (IN + υποερώτημα)

Q3 Εταιρείες που είχαν ζημιές σε τουλάχιστον μία χρήση (EXISTS)

Q4 Για κάθε εταιρεία: έσοδα 2025 και, σε νέα στήλη, τα συνολικά έσοδα της αγοράς το 2025 (υποερώτημα στο SELECT)

Q5 Η ημέρα του 2025 με τον υψηλότερο όγκο για κάθε μετοχή (συσχετισμένο υποερώτημα)

Q6 Ο κλάδος με τον μεγαλύτερο μέσο ROE 2025 (υποερώτημα στο FROM + TOP 1)

Έλεγχος: Q1 → 13 · Q2 → 3 · Q3 → 4 (DLPA, IKRC, ACRD, ESPR) · Q4 → σύνολο αγοράς 23.380,2 · Q5 → 24 γραμμές · Q6 → Τράπεζες

Άσκηση 2 · Χτίστε το «reporting layer»

35 λεπτά · τέσσερα views που θα χρησιμοποιήσουμε μέχρι το τέλος του εξαμήνου

`vw_CompanyProfile`	Εταιρεία + όνομα κλάδου + χώρα + υπάλληλοι `Companies, Sectors`
`vw_DailyPrices`	Ημερήσιες τιμές με Ticker, όνομα κλάδου και τιμή του δείκτη αγοράς την ίδια μέρα `StockPrices, Companies, Sectors, MarketIndex`
`vw_Fundamentals`	Οικονομικά στοιχεία ανά εταιρεία & έτος + δείκτες: NetMargin, ROE, Debt/Equity `Financials, Companies, Sectors`
`vw_DimDate`	Διάσταση ημερομηνίας: έτος, τρίμηνο, μήνας, ημέρα εβδομάδας `MarketIndex`



Έλεγχος: SELECT COUNT(*) σε κάθε view → 24 · 12.264 · 120 · 511 γραμμές. Αν βγάλετε άλλο αριθμό, ελέγξτε τα JOINs!

Ορόσημο: το project αρχίζει να παίρνει μορφή

Η ενότητα SQL σας έδωσε το «Βήμα 2» του end-to-end pipeline

1

Ερώτηση

Διαλέξτε ένα επιχειρηματικό ερώτημα για τη βάση

2

SQL

Γράψτε τα views που το απαντούν ✓

3

R & Οικονομετρία

εβδ. 6–12

4

Dashboard

εβδ. 11–13

Παραδείγματα ερωτημάτων για το project

- Ποιες μετοχές κινούνται περισσότερο από την αγορά (υψηλό beta) — και ανήκουν σε συγκεκριμένους κλάδους;
- Εξηγούν τα θεμελιώδη μεγέθη (ROE, μόχλευση) την απόδοση των μετοχών το 2025;
- Έχουν οι μετοχές με μέρισμα χαμηλότερη μεταβλητότητα;

Ανάθεση DataCamp · Εβδομάδα 5

Ολοκλήρωση πριν το επόμενο μάθημα · στο DataCamp Classroom του μαθήματος



Data Manipulation in SQL
Κεφ. 2 Short and Simple Subqueries

~1 ώρα



Database Design
Κεφ. 1 Processing, Storing & Organizing Data · Κεφ. 2 Database Schemas & Normalization · Κεφ. 3 Database Views

~3 ώρες



Προετοιμασία εβδ. 6 · Introduction to R
Κεφ. 1 Intro to basics — εγκαταστήστε R & RStudio

~30΄



Προαιρετικά: Data Manipulation in SQL, κεφ. 3 (correlated subqueries & CTEs) για όσους θέλουν να πάνε παραπέρα.

Σύνοψη ενότητας SQL & επόμενη εβδομάδα

Τι κρατάμε από σήμερα

- Υποερώτημα = ερώτημα σε παρενθέσεις: στο WHERE, SELECT ή FROM
- IN για λίστες · EXISTS για «υπάρχει;»
- ANY ≈ σύγκριση με MIN · ALL ≈ σύγκριση με MAX
- Συσχετισμένο υποερώτημα: «ο δικός μου μέσος όρος»
- View = αποθηκευμένο SELECT · GO, χωρίς ORDER BY
- Facts & διαστάσεις · star vs snowflake · 1NF–3NF

Εβδομάδα 6

- Νέα ενότητα: Οικονομετρία
- Απλή & πολλαπλή γραμμική παλινδρόμηση
- Το πρώτο μας μοντέλο: μετοχή vs αγορά (beta)
- Φέρτε εγκατεστημένα R + RStudio!